Reading the Poison: Relative-Entropy Detection of Word-Substitution Poisoning in a Cybersecurity RAG Knowledge Base

Ali Yousefi, Hafizul Asad, and Nikos Komninos

***Abstract*—In a retrieval-augmented generation (RAG) system the knowledge base is part of what has to be trusted. We study an attacker who edits existing passages by swapping words for synonyms (PWWS, TextFooler) in a small cybersecurity RAG system built from MITRE ATT&CK and CAPEC, using a reviewed 44-question benchmark, repeated paired runs and a control corpus in which only the layout changes. Poisoning lowered judged answer correctness from 0.714 to 0.555. The drop came from questions whose source section had been edited, and mostly from lost evidence and refusals rather than from false statements. Divergences over the whole passage were weak detectors, and some of them also responded to the layout-only control. A single term of the relative entropy, the log-ratio between a word and its most probable synonym, worked better: with a word-frequency reference it reached an AUROC of 0.82, and with a small language model as the reference 0.95, while staying near chance (0.53) on the control. It was clearly better than whole-passage perplexity and similar to a windowed perplexity filter, and it also names the word it suspects was replaced. Used as a retrieval filter it halved how often edited text reached the generator, but correctness did not recover. An attacker that also had to keep each passage at least as probable under the language model did not succeed on any of 57 sections. Finally, we tested a runtime monitor that checks answers against trusted MITRE STIX data and keeps a memory of the passages it rejects. On its own it reduced exposure to edited text (0.41 to 0.26) without restoring correctness. When it was allowed to add the trusted MITRE entry as a source, correctness rose to 0.786, which shows what a trusted source can add rather than what detection can. Checking answers for grounding after generation did not catch the incomplete answers this attack causes.**

***Index Terms*—Retrieval-augmented generation, data poisoning, adversarial text, relative entropy, runtime verification, large language models.**

I. Introduction

A RAG system answers a question by retrieving passages from a knowledge base and asking a language model to answer from them [1]. The answers can only be as reliable as those passages. Most work on poisoning RAG adds new passages written to steer answers towards a target [2], [3]. A quieter option for an attacker is to *edit* passages that are already there. Word-level adversarial attacks such as PWWS [4] and TextFooler [5] swap words for synonyms until a classifier changes its decision, and the edited text still reads almost normally. Ikbarieh et al. used TextFooler edits of this kind against the knowledge base of an LLM-based IoT security framework and found that its attack analysis and mitigation advice got worse [6].

This paper looks at that threat in a cybersecurity question-answering system and asks three questions. How much does word-substitution poisoning hurt the system, and how? Can a detector based on relative entropy find the edited passages, and what reference distribution does it need? And what happens when the attacker knows about the detector?

The starting point is relative entropy. If the word distribution p of a passage is compared with a reference q for normal text, $\mathrm{KL}(p \,\|\, q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}$ [7] might be expected to rise when words are swapped for unusual synonyms. In our data the full sum was a weak signal. One of its terms, the log-ratio between a word and its likely original under q, was much stronger once q was good enough. The idea is close to frequency-guided substitution detection [8]; we score text against a domain reference and then a language-model reference, without needing the attacked classifier.

The contributions are:

- A recorded testbed: clean, PWWS-poisoned and layout-only control knowledge bases, a reviewed benchmark with source anchors, and paired runs whose records are written once and checksummed (Section III).
- A measurement of how the attack does damage: mostly by pushing the answering evidence out of the top results and causing refusals, and only occasionally by text the model repeats as fact (Section V-A).
- A comparison of relative-entropy detectors, including perplexity baselines and the layout-only control, in which local language-model improbability works best and the synonym gap is the variant that can explain itself (Section V-B).
- Live runs of three retrieval-time filters and an answer-level evidence check (Sections V-C and V-F).
- Adaptive attackers with increasing knowledge and budget, a swap-level analysis of why they fail, and a second attack (Sections V-D and V-E).
- A runtime monitor with a quarantine memory, specified against trusted MITRE STIX data, with its costs and failure cases reported alongside its gains (Section V-G).

II. Related Work

Poisoning RAG. PoisonedRAG [2] injects passages that are retrieved for a target question and lead to a chosen answer; Zhong et al. [3] craft passages that are retrieved for many

The authors are with the Department of Computer Science, City St George's, University of London, London, UK.

queries. Both add text. Ikbarieh et al. [6] edit an existing knowledge base with TextFooler. We also study in-place edits, and we measure where between retrieval and generation the damage happens.

Word-substitution attacks and their detection. PWWS [4] ranks words by saliency and replaces each with the WordNet [9] synonym that most lowers the classifier's confidence. TextFooler [5] takes substitutes from counter-fitted word embeddings. Both are available in TextAttack [10]. FGWS [8] detects such examples by replacing infrequent words with frequent synonyms and checking whether the classifier's output changes. We keep FGWS's observation that adversarial substitutes tend to be rarer than the words they replace, but we score the text against a reference distribution instead of querying the attacked classifier, so the check can run at retrieval time.

Perplexity filters. Language-model perplexity is a common model-agnostic filter for adversarial text. Adversarial suffixes have very high perplexity [11], and windowed perplexity is a strong baseline against them [12]. RAGuard [13] widens retrieval and filters passages by chunk-wise perplexity. We compare against whole-passage and windowed perplexity computed with the same language model as our detector.

Defending RAG. RobustRAG [14] answers from each passage separately and aggregates the answers, with certifiable guarantees. TrustRAG [15] clusters retrieved passages to find suspicious groups. Corrective RAG [16] grades retrieval and falls back to other sources when it is poor. RAGAS faithfulness [17] checks whether an answer follows from its sources, and SelfCheckGPT [18] uses sampling to find hallucinations. These defences are mainly aimed at injected passages or at answer quality. We filter *edited* passages before generation, which needs no change to the generator.

III. Threat Model and Testbed

System. A FastAPI backend retrieves the top 3 passages from Qdrant (all-MiniLM-L6-v2 embeddings; 500-character chunks with 100 characters of overlap) and answers with gpt-4o-mini at temperature 0, using a fixed prompt that tells the model to answer only from the context. A web interface shows clean and poisoned answers side by side and lets the user choose each defence.

Knowledge base. 18 documents of MITRE ATT&CK and CAPEC material on 14 attack types in 5 categories (DDoS, injection, malware, reconnaissance, spoofing), split into 200 passages.

Attacker. The attacker can edit text in the knowledge base. It cannot change the questions, the retrieval or generation settings, or the evaluation. We run PWWS (TextAttack) against a DistilBERT classifier that predicts the attack type of a section and was trained on the same material. The classifier is a *surrogate*: a section is rewritten when PWWS flips its label. Of 88 eligible sections, 31 are already misclassified and are skipped, and PWWS succeeds on 38 of the remaining 57. All adaptive experiments and their matched control use a retrained surrogate, on which plain PWWS succeeds on 39. The original and adversarial text of every section is kept in an attack manifest.

Layout-only control. The poisoning script also flattens the layout of the sections it rewrites. A control corpus applies the same flattening to the same sections but keeps the original words. Retrieval on the control is the same as on the clean corpus, so the layout change alone does not explain the damage.

Benchmark and protocol. There are 44 reviewed questions. Each has atomic reference facts and an exact source anchor, and development and held-out questions come from different source documents (35 and 9). Each question is run 3 times per condition, paired by schedule index, with the order of conditions alternated. An LLM judge [19] scores correctness against the reference facts and faithfulness against the retrieved passages, using a fixed rubric. On 75 stratified answers that the first author graded blind, the judge's correctness scores agreed well with the manual grades (quadratic weighted $\kappa = 0.82$). Retrieval relevance is Hit@3 under a section-level rule. *Edited-text exposure* records whether any attacker-edited word reached the generator. Retrieval returned the same passages in all three repetitions for every question, so the question is the unit of analysis: we report question-level cluster-bootstrap 95% intervals (10,000 resamples) and two-sided Wilcoxon signed-rank tests. Every run writes JSON records that are never overwritten, plus a SHA-256 completion manifest.

IV. Relative-Entropy Detectors

Let a passage contain words $w_1, \ldots, w_n$ with empirical distribution p, and let q be a reference distribution.

Whole-passage divergences. $\mathrm{KL}(p \,\|\, q)$ [7], the Jensen–Shannon divergence [20], and mean surprisal $-\frac{1}{n}\sum_i \log_2 q(w_i)$. We also report KL over letter frequencies, the usual textbook example of relative entropy.

Synonym gap. For each content word w with WordNet synonyms $S(w)$,

$$g(w) = \max_{s \in S(w)} \big[\log_2 q(s) - \log_2 q(w)\big], \tag{1}$$

which is the number of bits by which the most probable synonym beats the word that was actually used. It is the log-ratio term of KL, evaluated where a substitution would have happened. For example, the PWWS substitute "abnegate" gets a high score because its likely original, "deny", is far more probable. A passage's score is the mean of its three largest gaps.

References. (i) General English word frequencies. (ii) A *domain* reference: word counts from the rest of the knowledge base with the passage's own document left out, interpolated 50/50 with English. It does not need a clean copy of the poisoned document. (iii) The same counts with a Dirichlet prior, $q(w) = \frac{c(w)+\mu q_{\mathrm{en}}(w)}{N+\mu}$ with $\mu = 2000$ [21]. (iv) A *contextual* reference from a small causal language model, distilgpt2 [22]. For (iv) we take the 10 words with the largest domain gap and, for each, its 5 most probable synonyms s, and compute

$$g_{\mathrm{ctx}}(w, s) = \log_2 p_{\mathrm{LM}}(\text{passage}, w \to s) - \log_2 p_{\mathrm{LM}}(\text{passage}). \tag{2}$$

TABLE I
EFFECT OF PWWS POISONING (44 QUESTIONS, 132 PAIRED ANSWERS PER CONDITION). Δ: CHANGE IN CORRECTNESS.

Stratum	Hit@3	Correctness	Δ [95% CI]
All	.705→.568	.714→.555	−.159 [−.254, −.068]
Source section edited (27)	.630→.444	.682→.432	−.250 [−.383, −.124]
Not edited (17)	.824→.765	.765→.750	−.015 [−.083, .069]

The passage score is again the mean of the three largest gaps.

Perplexity baselines. With the same language model: *mean surprisal* ($\log_2$ perplexity of the whole passage) and *top-5 surprisal*, the mean of the five most surprising tokens, which is a windowed filter in the spirit of [12].

All scores use only information a defender would have; the attack labels are used only for evaluation. Thresholds were set once on development documents at a 10% false-positive rate and were not re-tuned on held-out documents or on live answers.

V. RESULTS

A. The attack and how it does damage

The drop in correctness comes from questions whose source section was edited (Table I). On questions answered from untouched sections of the same poisoned corpus the change was small and not significant. Together with the layout-only control, this points to the word substitutions as the cause. Most of the damage came from lost evidence. Refusals rose from 26 to 45 of 132 answers. Of the 48 answers whose correctness fell by at least 0.25, 27 were refusals and 31 had no relevant passage in the top 3. Wrong answers (not refusals, correctness below 0.25) rose only from 0 to 2. Both were to the same question, and in both the model repeated a substituted phrase, "Engraft Malicious Package" (originally "implants malicious software"), as if it were the name of an attack stage.

B. Detection

Table II and Fig. 1 show four things.

Whole-passage measures are weak. Letter frequencies keep their English statistics when words are swapped, so letter KL is below chance. Word KL against the domain reference does better (0.72–0.83), but it also scores 0.62 on the control, where no word changed, so part of what it picks up is which sections were attacked rather than the edits. Over a whole passage, the few changed words are diluted among about eighty others.

The per-word log-ratio works, and the reference matters more than the divergence or the smoothing. Dirichlet smoothing and interpolation give almost the same result (0.83 vs 0.82).

The contextual reference is better and cleaner on PWWS. It improves on the word-frequency gap on both PWWS corpora (paired differences +0.13 [+0.03, +0.21] and +0.09 [+0.03, +0.16]) and is close to chance on the control, where the word-frequency gap still scores 0.67. On TextFooler the two are not distinguishable (+0.02 [−0.05, +0.07]). On the 9 held-out documents (45 passages, 10 of them edited), the word-frequency gap scored higher than the contextual one (0.94 vs 0.88), so we do not claim the contextual reference is better everywhere.

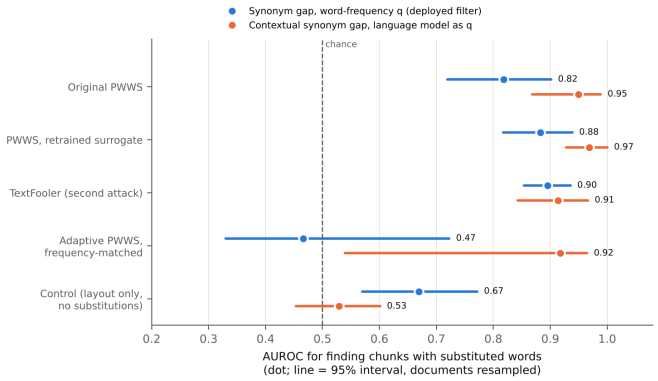


Fig. 1. Word-frequency and contextual synonym gap on every corpus, with document-clustered 95% intervals.

What helps is looking locally. The contextual gap beats whole-passage perplexity on PWWS and TextFooler (paired differences +0.18 [+0.11, +0.26] and +0.10 [+0.03, +0.16]), but it is statistically indistinguishable from windowed perplexity (+0.04 [−0.003, +0.08] on PWWS, −0.01 [−0.07, +0.03] on TextFooler). What the synonym gap adds over a windowed perplexity filter is an explanation. Of the 27 edited passages the word-frequency gap flags, 22 name the exact replaced word and its likely original (for example abnegate→deny, meshing→network). At the fixed thresholds the contextual gap also had a slightly lower false-positive rate than windowed perplexity on every attacked corpus (for example 10.4% vs 11.8% on PWWS).

C. Live defences

Each defence retrieves 10 candidates, scores them, and passes 3 to the generator. *Drop* removes candidates above the word-frequency threshold (7.23 bits). *Repair* first replaces each word whose gap is above 9.5 bits by its likely original and keeps the passage if it then scores under the threshold. *Contextual* drops candidates using the contextual gap (4.46 bits). Each defence was run live on the full benchmark on both the clean and the poisoned corpus (792 records per run).

All three defences removed the two wrong answers (Table III). The contextual filter halved how often edited text reached the model and had the smallest cost on clean data; it flagged 8.4% of clean candidates, against 10.9% for the word-frequency filter. None of the filters restored correctness, and the contextual filter lowered it further. When the passage that answers a question is the edited one, removing it leaves the model without evidence, and the model then refuses rather than guesses. Repair did not help either, because word-level WordNet repair restores too few substitutions (recall 0.12 at precision 0.89). In short, filtering made the poisoned knowledge base safer to read but did not make it useful again.

D. Adaptive attackers

A first-order attacker that only swaps to words at least as frequent under the defender's q evades the word-frequency

TABLE II
DETECTION OF PASSAGES THAT CONTAIN SUBSTITUTED WORDS (AUROC). BRACKETS: 95% BOOTSTRAP INTERVALS THAT RESAMPLE DOCUMENTS. 191 PASSAGES (193 FOR TEXTFOOLER); EDITED PASSAGES: 47 PWWS, 59 PWWS ON THE RETRAINED SURROGATE, 82 TEXTFOOLER, 13 FREQUENCY-MATCHED ADAPTIVE. THE CONTROL HAS NO WORD EDITS: ITS "POSITIVES" ARE THE 48 PASSAGES WHOSE LAYOUT WAS FLATTENED, SO ANY SCORE ABOVE 0.5 THERE REFLECTS LAYOUT OR SECTION CHOICE RATHER THAN SUBSTITUTIONS. LAST ROW: EDITS FLAGGED AT THE CONTEXTUAL THRESHOLD (4.46 BITS) FIXED ON PWWS DEVELOPMENT DOCUMENTS, WHICH FLAGS 9–14% OF UNEDITED PASSAGES.

Detector	PWWS	PWWS (retrained)	TextFooler	Control (layout only)	Frequency-matched adaptive
Letter-frequency KL	0.33	0.31	0.28	0.35	0.36
Word KL vs domain	0.72	0.74	0.83	0.62	0.50
Jensen–Shannon vs domain	0.47	0.47	0.48	0.43	0.44
Mean surprisal (perplexity)	0.77	0.77	0.82	0.45	0.73
Top-5 surprisal (windowed perplexity)	0.91	0.93	0.92	0.56	0.85
Synonym gap, domain q	0.82 [.72, .90]	0.88 [.82, .94]	0.90 [.85, .94]	0.67 [.57, .77]	0.47 [.33, .72]
Synonym gap, Dirichlet q	0.83	0.89	0.90	0.68	0.48
Contextual synonym gap	0.95 [.87, .99]	0.97 [.93, 1.0]	0.91 [.84, .97]	0.53 [.45, .60]	0.92 [.54, .96]
Edits flagged, fixed contextual threshold	42/47	54/59	70/82	–	10/13

TABLE III
LIVE DEFENCES ON THE POISONED CORPUS (44 QUESTIONS × 3; p: WILCOXON AGAINST NO DEFENCE). CLEAN COST: CHANGE IN CORRECTNESS ON THE CLEAN CORPUS.

Defence	Exposure	Correct.	Wrong	Refusals	Clean cost
None	.409	.555	2	45	–
Drop (freq.)	.273[a]	.526	0	53	−.064
Repair	.364	.506	0	54	−.042
Drop (ctx.)	.204[b]	.485[c]	0	63	−.013

[a] $p = 0.031$; [b] $p = 0.012$; [c] $p = 0.020$.

TABLE IV
ADAPTIVE ATTACKERS (SAME RETRAINED SURROGATE AND 57 ATTACKABLE SECTIONS; ONLY THE EVASION RULE DIFFERS). AUROC OF THE WORD-FREQUENCY AND CONTEXTUAL SYNONYM GAPS.

Attacker	Sections	Edits	Freq.	Ctx.
Unrestricted PWWS	39	59	.88	.97
1st order: frequency-matched	8	13	.47	.92
2nd order, strict	0	0	–	–
2nd order, ≤4.46 bits	0	0	–	–
2nd order, ≤20 bits	0	0	–	–
2nd order, ≤100 bits	5	6	.51	.78
2nd order, ≤300 bits	8	13	.47	.92

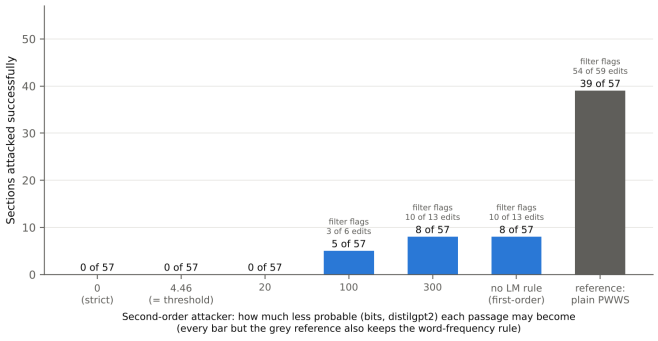


Fig. 2. Second-order attacker: successful sections as the allowed loss of language-model likelihood grows, with the share of resulting edits flagged by the fixed contextual filter.

detector, as intended (Table IV). The contextual detector still finds most of its edits, because the attacker matched how common its words are and not whether they fit the sentence. Evasion also costs the attacker: it succeeds on 8 sections instead of 39.

A second-order attacker that must also keep each passage at least as probable under the detector's language model succeeded on no section. To see why, we looked at the 550 swaps that unrestricted PWWS used in its 39 successful attacks. 109 of them (20%) satisfy the frequency rule, only 9 (1.6%) leave the passage at least as probable under the language model, and 5 satisfy both. The median swap makes its passage 17.9 bits less probable. Our reading is that natural source text sits close to a local maximum of language-model likelihood, and the swaps that move a classifier tend to move the text away from it.

We then relaxed the rule (Fig. 2). With an allowed loss of up to 4.46 bits (the contextual threshold) or 20 bits, the attacker still succeeded on no section. At 100 bits it succeeded on 5 sections; the contextual detector scored their 6 edited passages at an AUROC of 0.78, and the fixed filter flagged 3 of the 6, so 3 edited passages got through. At 300 bits the language-model rule no longer constrains the attacker, and the attack produced exactly the same corpus as the frequency-matched attacker, whose edits the contextual filter flags 10 of 13. So in this family of attackers, those that respected the detector's rule did not succeed, and those that succeeded left most, but not all, of their edits detectable. These conclusions rest on 6 and 13 edited passages.

E. A second attack

TextFooler takes substitutes from word embeddings rather than WordNet. With the same surrogate and sections it was the stronger attack: 49 of 57 sections and 82 edited passages. Both synonym-gap detectors worked on it without changes (Table II). At the thresholds fixed on PWWS development documents, the contextual filter flagged 70 of 82 edits (85%) at a 9% false-positive rate, and the word-frequency filter 43 of 82 (52%) at 6%.

Run live with the same thresholds, TextFooler also reached the generator more often: edited text was in the context for 82% of questions (PWWS: 41%), and correctness fell

from 0.714 to 0.564 ($p = 0.002$), again through refusals rather than wrong answers. The word-frequency filter barely changed exposure (82% to 80%); when most candidates are edited, dropping some of them mainly promotes others. The contextual filter cut exposure to 52% ($p < 0.001$; against the word-frequency filter, $p = 0.003$), with the same cost as on PWWS: correctness fell further, to 0.447, and refusals rose from 54 to 74 of 132.

F. Checking answers instead of passages

We also tried an answer-level check. A separate verifier call splits each draft answer into claims, judges each claim against the retrieved passages, and must quote its evidence word for word (the quotes are checked in code). The answer is withheld unless every claim is supported. We replayed it on 352 recorded answers (one repetition of every question in eight conditions). It was not a useful error detector here. Of the 238 answers that were not refusals, 38% were not fully correct, but only 28% of the answers it flagged were wrong (precision 0.28 [0.13, 0.48]). It flagged 20% of the incorrect answers and 31% of the correct ones. Substitution poisoning mostly removes evidence, so the typical error is an incomplete answer in which every statement is grounded: 68 of 89 partial answers passed. It did withhold the one wrong answer in that sample. Against in-place substitution, the check has to act on passages before generation; a grounding check afterwards cannot see what is missing.

G. A runtime monitor with feedback and memory

The filters above forget: the same poisoned passage is scored again each time it is retrieved. We therefore tried a runtime monitor, in the spirit of runtime verification, that checks each draft answer, removes the passages it blames, and remembers them. Its trusted reference is the official MITRE STIX data for CAPEC and ATT&CK. The files are pinned by SHA-256, and the monitor refuses to start if they change. The attacker in our threat model edits the knowledge base, not these files.

Each draft answer is checked against three properties. φ_1: every rare word in the answer (Zipf frequency below 3.5) that was copied from the retrieved passages either occurs in the trusted MITRE vocabulary or in the question. φ_2: the answer is not a refusal when the trusted reference covers the attack being asked about. φ_3, a temporal safety property: $\mathbf{G}(\mathit{quarantined}(p) \rightarrow \mathbf{G}\,\neg \mathit{inContext}(p))$, that is, once a passage is quarantined it never appears in the context again. φ_1 and φ_2 are checked by Python code on each answer; φ_3 holds by construction, because quarantined passages are skipped at retrieval, and we also checked it on every recorded trace.

When φ_1 or φ_2 is violated, the monitor blames the retrieved passages that contain rare words outside the trusted vocabulary, adds them to an append-only memory keyed by a hash of their source and text, retrieves again without them, and asks the model again with the rejected answer and the violated property as feedback, at most twice. In *verify* mode an unresolved φ_1 withholds the answer and an unresolved φ_2 releases the refusal. In *restore* mode an unresolved φ_2 instead adds the trusted MITRE entry for the attack as an extra source. Restore mode therefore gives the model information the clean system never sees, and we treat it as an upper bound on what a trusted source can add rather than as a defence comparable with the filters. Each condition starts with an empty memory, and repetitions 2 and 3 meet the same attack again.

TABLE V
RUNTIME MONITOR, LIVE, SECOND VERSION (44 QUESTIONS × 3; QUESTION-LEVEL PAIRED COMPARISONS; p: WILCOXON). ONE TEXTFOOLER ANSWER WAS LOST TO A NETWORK ERROR AND IS EXCLUDED.

Comparison	Monitor	Baseline	Δ [95% CI]
Verify, PWWS: exposure	.258	.409	−.152 [−.265, −.061][a]
Verify, PWWS: correctness	.515	.555	−.040 [−.085, .000]
Verify vs ctx. filter: exposure	.258	.204	+.053 [−.030, .144]
Verify vs ctx. filter: correctness	.515	.485	+.030 [−.002, .068]
Verify, clean data: correctness	.708	.714	−.006 [−.023, .011]
Verify, TextFooler: exposure	.583	.818	−.235 [−.356, −.114][b]
Verify, TextFooler: correctness	.542	.564	−.023 [−.087, .047]
Restore, PWWS: correctness	.786	.555	+.231 [.136, .335][c]
Restore vs clean: correctness	.786	.714	+.072 [−.004, .152]

[a] $p = 0.016$; [b] $p = 0.001$; [c] $p = 0.0001$.

The first version of φ_1 flagged any rare word outside the trusted vocabulary, including words the model chose itself. In a first live run it withheld 10 answers, one of them on clean data, partly because of words from the knowledge base's OWASP and NIST material, which the MITRE files do not contain. We changed φ_1 to count only words copied from the retrieved passages and repeated the full run; Table V reports the second run. Both runs are kept.

In verify mode the monitor reduced exposure to edited text on both attacks (Table V) and left no wrong answers on PWWS (2 without it). It did not restore correctness, which was slightly lower than without the monitor, and refusals rose from 45 to 57. Against the contextual filter it was not significantly different in either exposure or correctness. On clean data it cost almost nothing, quarantined no passage and withheld no answer, although one clean answer was judged wrong where the unmonitored system had none. The substituted phrase "Engraft Malicious Package", which reached the user in 3 of 3 repetitions without a defence, did not appear in any repetition with the monitor. Because it was already absent in the first repetition, this is the effect of checking and retrying, not of the memory, and the plain filters also removed it.

The memory behaved as specified: φ_3 held on every recorded trace in all four conditions. Quarantined passages were skipped on later questions, so the number skipped at the first retrieval rose from 0.73 to 1.38 per answer between repetition 1 and repetitions 2–3 on PWWS (1.25 to 2.81 on TextFooler), and on TextFooler the number of generations per answer fell from 1.32 to 1.01. The effect on exposure was small (0.27 to 0.25 on PWWS, 0.60 to 0.57 on TextFooler), and correctness was lower in the later repetitions on both attacks (0.53 to 0.51 and 0.59 to 0.52), because more passages had been removed. The memory is also only as accurate as the blame step. On TextFooler 26 of 27 quarantined passages contained edited text, but on PWWS only 10 of 18 did. All quarantines in this run were triggered by φ_2, that is, by refusals, and the passages blamed were those with rare

words outside the MITRE vocabulary; ordinary words such as "ingested" were enough to quarantine an unedited passage.

With the trusted fallback, correctness rose from 0.555 to 0.786 and refusals fell from 45 to 7; the monitor fell back in 58 of 132 answers. This is numerically above the unattacked system (0.714), and the difference is not significant. The gain comes from the trusted MITRE entry, which covers most of our questions because the knowledge base was built from MITRE material. It would be smaller for content without a trusted counterpart. The monitor's part is deciding when to fall back and which passages to stop using.

The monitor has clear limits. φ_1 can be evaded by an attacker who chooses substitutes inside the trusted vocabulary or common enough to fall above the rarity cut-off. The trusted reference covers only MITRE, so OWASP and NIST content cannot be checked. And because blame is based on rare words, an attacker could try to get clean passages quarantined; we saw this happen by accident on PWWS, where 8 of 18 quarantined passages were unedited.

VI. Discussion

Why the log-ratio and not the sum. $\mathrm{KL}(p \,\|\, q)$ is a sum of log-ratio terms weighted by p. A substitution changes only a few terms, each with a small weight, and the passage's topic already makes p differ from q by much more. The synonym gap looks at the log-ratio only where an attack would act, comparing each word with the word it may have replaced.

Why a better q mattered more than a better divergence. Changing the smoothing or using a symmetric divergence made little difference, while moving from a unigram to a contextual q made a large one on PWWS. A unigram reference can only say that a word is rare; a contextual one can say that it is unlikely in this sentence. The perplexity baselines point the same way: what matters is measuring improbability locally, where the edit is.

Detection is not restoration. In this corpus the passage that answers an edited question is usually the edited one, so a filter that removes poison also removes evidence. Getting correctness back needs another source of the same information, such as the trusted MITRE entry used by the monitor's restore mode, or the original text, which a masked language model conditioned on the domain might help recover.

Detection constrains the attacker. Each extra constraint needed to evade a better reference removed most of the swaps that make the attack work. Against the contextual detector, the strict second-order attacker found no successful attack in our setting. This is evidence about one attacker family, not a guarantee.

VII. Limitations

The study uses one small knowledge base (18 documents), one generation model, one embedding model and 44 questions, of which only 9 are held out; the adaptive results rest on 13 and 6 edited passages, and the confidence intervals are wide. The second-order attacker constrains whole-passage likelihood but does not optimise against the detector's score directly; an attacker that does, a masked-language-model attack, or a larger reference model could behave differently. The attack targets a surrogate classifier, not the RAG pipeline itself; its effect on retrieval and answers is measured, not optimised. Correctness is scored by an LLM judge. On a blind manual check of 75 stratified answers by the first author it agreed well with the manual grades ($\kappa = 0.82$, Spearman $\rho = 0.85$) but was more lenient by 0.08 on average, by 0.02 on clean answers, 0.08 on poisoned answers and 0.14 on answers produced with the word-frequency filter. A second, independent grader would make this check stronger. Thresholds were fixed once on development data, and the balance between safety and utility depends on them. The monitor's limitations are discussed in Section V-G.

VIII. Ethics

The attacks edit a local copy of public MITRE material. No real system is attacked and no malware is run. No personal data is used; question and passage text is sent to a commercial LLM API. Detection scores are evidence for a human to review, not proof that a passage is clean or that an answer is true.

IX. Reproducibility

Code, corpora, attack manifests, run records and analysis scripts are at https://github.com/Aliyousefi9403/dissertation-RAG. Run records are JSON files that are written once and never overwritten, with a SHA-256 completion manifest per run. A verification script re-hashes all run records and checks the main numbers reported here against the result files that produced them.

Acknowledgment

The relative-entropy approach was suggested by N. Komninos, and the runtime-verification monitor by H. Asad. AI tools were used to help with coding, analysis and drafting; the first author checked all code, numbers and references.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474.

[2] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proc. 34th USENIX Security Symposium*, 2025.

[3] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, "Poisoning retrieval corpora by injecting adversarial passages," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 13 764–13 775.

[4] S. Ren, Y. Deng, K. He, and W. Che, "Generating natural language adversarial examples through probability weighted word saliency," in *Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 1085–1097.

[5] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is BERT really robust? A strong baseline for natural language attack on text classification and entailment," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 34, no. 5, 2020, pp. 8018–8025.

[6] S. Ikbarieh, K. Aryal, and M. Gupta, "RAG-targeted adversarial attack on LLM-based threat detection and mitigation framework," arXiv:2511.06212, 2025.

[7] S. Kullback and R. A. Leibler, "On information and sufficiency," *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.

[8] M. Mozes, P. Stenetorp, B. Kleinberg, and L. Griffin, "Frequency-guided word substitutions for detecting textual adversarial examples," in *Proc. 16th Conference of the European Chapter of the ACL (EACL)*, 2021, pp. 171–186.
[9] G. A. Miller, "WordNet: A lexical database for English," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.
[10] J. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 119–126.
[11] G. Alon and M. Kamfonas, "Detecting language model attacks with perplexity," arXiv:2308.14132, 2023.
[12] N. Jain *et al.*, "Baseline defenses for adversarial attacks against aligned language models," arXiv:2309.00614, 2023.
[13] Z. Cheng, J. Sun, A. Gao, Y. Quan, Z. Liu, X. Hu, and M. Fang, "Secure retrieval-augmented generation against poisoning attacks," arXiv:2510.25025, 2025.
[14] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, "Certifiably robust RAG against retrieval corruption," arXiv:2405.15556, 2024.
[15] H. Zhou, K.-H. Lee, Z. Zhan, Y. Chen, Z. Li, Z. Wang, H. Haddadi, and E. Yilmaz, "TrustRAG: Enhancing robustness and trustworthiness in retrieval-augmented generation," arXiv:2501.00879, 2025.
[16] S.-Q. Yan *et al.*, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024.
[17] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. EACL: System Demonstrations*, 2024, pp. 150–158.
[18] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. EMNLP*, 2023, pp. 9004–9017.
[19] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.
[20] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.
[21] C. Zhai and J. Lafferty, "A study of smoothing methods for language models applied to information retrieval," *ACM Transactions on Information Systems*, vol. 22, no. 2, pp. 179–214, 2004.
[22] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.